Departamento de Informática

Lab04

Leonor Rebola

Adriana Abreu

Faculdade de Engenharia
Departmento de Informática
Licenciatura em Inteligência Artificial e Ciência de Dados

Covilhã, Março 2026

Página intencionalmente deixada em branco.

Departamento de Informática

Lab04

Leonor Rebola

Nº Estudante 53663

Adriana Abreu

Nº Estudante 53672

Professor: Sebastião Pais
Professor de Extração e Transformação de Dados,
Universidade da Beira Interior

Faculdade de Engenharia
Departmento de Informática
Licenciatura em Inteligência Artificial e Ciência de Dados

Covilhã, Março 2026

Página intencionalmente deixada em branco.

Conteúdo

Conteúdo 4

Lista de Figuras 5

1 Pré Processamento dos Dados 6

2 TF-IDF, BoW e Word2Vec 8

3 Bert 12

4 Gráficos 14

Lista de Figuras

4.1 Comparação entre accuracies 14
4.2 Comparação entre F1 Scores 14

1

Pré Processamento dos Dados

No pré processamento dos Dados (utilizámos um csv do kaggle, sobre tweets: Link kaggle

Este csv contém três colunas: sentiment(se é positivo ou negativo), time (a data do tweet) e tweet (o texto em si do tweet)

Para o pré processamento, só mexemos na coluna tweet que era a única que precisava de ser tratada e normalizada. Fizemos a Tokenização, ou seja separar por espaços/pontuação, gerando uma lista de palavras, em seguida remoção das stop words e por fim a normalização, isto tudo utilizando a bilioteca **nltk**

Abaixo estáo código utilizado:

```
import pandas as pd
import nltk
import string
from nltk.corpus import stopwords
from nltk.tokenize import word_tokenize

def main():
    nltk.download('punkt')
    nltk.download('stopwords')

    df = pd.read_csv("tweet_sub.csv")

    stop_words = set(stopwords.words('portuguese'))

    def preprocessar(texto):
        texto = texto.lower()
        texto = texto.translate(str.maketrans('', '', string.
            punctuation))
        tokens = word_tokenize(texto)
        tokens = [t for t in tokens if t not in stop_words]
        return tokens

    df["tokens"] = df["tweet"].apply(preprocessar)
    df["texto_limpo"] = df["tokens"].apply(lambda x: " ".join(x))
```

```
    return df


if __name__ == "__main__":
   df =main()
   print(df["texto_limpo"].iloc[0])
```

2

TF-IDF, BoW e Word2Vec

Para determinar o resultado seguindo o método TF-IDF e Bag of Words (BoW) foi usado o seguinte código

```
from processamento import main
from sklearn.feature_extraction.text import CountVectorizer,
    TfidfVectorizer
from sklearn.model_selection import train_test_split
from sklearn.pipeline import Pipeline
from sklearn.naive_bayes import MultinomialNB
from sklearn.metrics import accuracy_score, f1_score

def tfidf_bow():
    df = main()

    # Bag of Words
    vectorizer_bow = CountVectorizer()
    X_bow = vectorizer_bow.fit_transform(df["texto_limpo"])

    print("BoW vocab:", len(vectorizer_bow.vocabulary_))
    print("BoW shape:", X_bow.shape)

    # TF-IDF
    vectorizer_tfidf = TfidfVectorizer()
    X_tfidf = vectorizer_tfidf.fit_transform(df["texto_limpo"])

    print("TF-IDF shape:", X_tfidf.shape)

    # Train-test split
    X_train, X_test, y_train, y_test = train_test_split(
        df["texto_limpo"], df["sentiment"], test_size=0.2,
            random_state=42
    )

    # Pipeline com TF-IDF + Naive Bayes
    model = Pipeline([
```

```
            ("vectorizer", TfidfVectorizer()),
            ("classifier", MultinomialNB())
        ])

        model.fit(X_train, y_train)
        y_pred = model.predict(X_test)

        # métricas
        acc = accuracy_score(y_test, y_pred)
        f1 = f1_score(y_test, y_pred, average="weighted")

        print(f"Accuracy: {acc:.4f}")
        print(f"F1: {f1:.4f}")

        return acc, f1


    if __name__ == "__main__":
        tfidf_bow()
```

Em que o output gerado foi este:

- BoW vocab: 4941
- BoW shape: (1500, 4941)
- TF-IDF shape: (1500, 4941)
- Accuracy: 0.7167
- F1: 0.7165

Já para a técnica Word2Vec, foi usado o seguinte código

```
    from processamento import main
    import numpy as np
    from gensim.models import Word2Vec
    from sklearn.model_selection import train_test_split
    from sklearn.metrics import accuracy_score, f1_score
    from sklearn.naive_bayes import GaussianNB

    def word2vec():
        df = main()

        # treinar Word2Vec
        model_w2v = Word2Vec(
            sentences=df["tokens"],
            vector_size=100,
            window=5,
            min_count=2,
            workers=4
        )
```

```
    # função para vetorizar um documento
    def document_vector(tokens):
        vectors = [model_w2v.wv[word] for word in tokens if word
            in model_w2v.wv]

        if len(vectors) == 0:
            return np.zeros(model_w2v.vector_size)

        return np.mean(vectors, axis=0)

    # transformar todos os tweets em vetores
    X_w2v = np.array(df["tokens"].apply(document_vector).tolist()
        )
    print("Word2Vec shape:", X_w2v.shape)

    # split train/test
    X_train, X_test, y_train, y_test = train_test_split(
        X_w2v, df["sentiment"], test_size=0.2, random_state=42
    )

    # classificador
    model = GaussianNB()
    model.fit(X_train, y_train)

    # previsões
    y_pred = model.predict(X_test)

    # métricas
    acc = accuracy_score(y_test, y_pred)
    f1 = f1_score(y_test, y_pred, average="weighted")

    print(f"Accuracy: {acc:.4f}")
    print(f"F1: {f1:.4f}")

    return acc, f1

if __name__ == "__main__":
    word2vec()
```

Em que o output gerado foi este:

- Word2Vec shape: (1500, 100)
- Accuracy: 0.5267
- F1: 0.5114

De entre estas três técnicas, o TF-IDF mostrou melhores resultados que o BoW. Esta diferença de desempenho pode ser explicada pelas características do dataset e das próprias representações. O TF-IDF baseia-se na frequência das palavras e é particularmente eficaz em textos curtos, como tweets, onde a presença de determinadas palavras é altamente indicativa da classe. Já o Word2Vec aprende representações semânticas das palavras com base no contexto. No entanto, este tipo de abordagem requer grandes volumes de dados para produzir embeddings de alta qualidade. Como o dataset utilizado contém apenas 1500 exemplos, o modelo não consegue capturar adequadamente as relações semânticas entre palavras.
Em conclusão, para este dataset específico, **o TF-IDF revelou-se mais adequado para a tarefa de classificação**. No entanto, o Word2Vec continua a ser uma abordagem relevante, especialmente em cenários com maiores volumes de dados ou quando se pretende capturar relações semânticas mais profundas entre palavras.

3

Bert

Para fazer uso desta abordagem, usou-se o seguinte código

```
from processamento import main

from transformers import AutoTokenizer, AutoModel
import torch
import numpy as np

from sklearn.model_selection import train_test_split
from sklearn.metrics import accuracy_score, f1_score
from sklearn.linear_model import LogisticRegression



df = main()

print("\nA gerar embeddings com BERT...")

model_name = "neuralmind/bert-base-portuguese-cased"

tokenizer = AutoTokenizer.from_pretrained(model_name)
bert_model = AutoModel.from_pretrained(model_name)

def bert_embedding(text):

    inputs = tokenizer(
        text,
        return_tensors="pt",
        truncation=True,
        padding=True,
        max_length=128
    )

    with torch.no_grad():
        outputs = bert_model(**inputs)
```

```
    embedding = outputs.last_hidden_state[:,0,:]

    return embedding.squeeze().numpy()


df["bert_vector"] = df["tweet"].apply(bert_embedding)


X_bert = np.vstack(df["bert_vector"].values)

    # dividir dados
X_train, X_test, y_train, y_test = train_test_split(
    X_bert, df["sentiment"], test_size=0.2, random_state=42
    )

# modelo simples de classificação
clf = LogisticRegression(max_iter=1000)

clf.fit(X_train, y_train)

y_pred = clf.predict(X_test)

print("\nResultados BERT")
print("Accuracy:", accuracy_score(y_test, y_pred))
print("F1:", f1_score(y_test, y_pred, average="weighted"))
```

Em que o output gerado foi o seguinte:

- Accuracy: 0.6366666666666667
- F1: 0.6365253311104937

O BERT é um modelo baseado em redes neuronais profundas que permite capturar o contexto das palavras de forma bidirecional, produzindo representações mais ricas do que métodos tradicionais como TF-IDF ou Word2Vec. No entanto, neste caso, **o desempenho não ultrapassou o TF-IDF**. Apesar de ser um modelo mais avançado, a sua utilização sem ajuste (fine-tuning) limita o seu potencial, sendo usado apenas como extrator de features, seguido de um classificador simples (Logistic Regression), o que pode não ser suficiente para tirar partido completo da arquitetura.
Em conclusão, embora o BERT permita obter representações contextuais mais sofisticadas, o TF-IDF revelou-se mais eficaz para esta tarefa específica. No entanto, espera-se que o desempenho do BERT melhore significativamente caso seja aplicado fine-tuning diretamente para a tarefa de classificação.

4

Gráficos

Para podermos comparar as quatro técnicas usadas, fizemos gráficos para comparar a accuracy e F1.

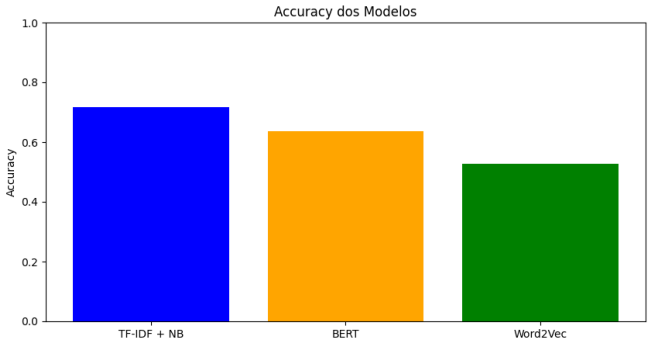

Figura 4.1: Comparação entre accuracies

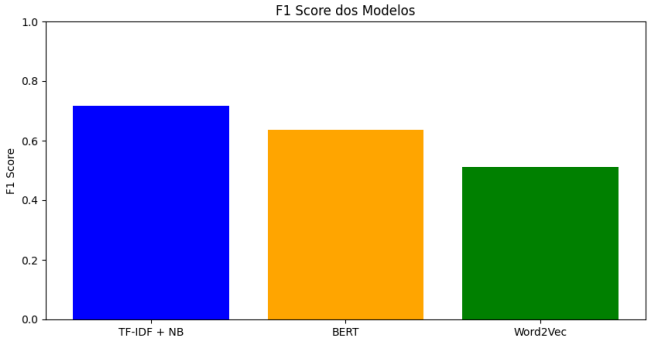

Figura 4.2: Comparação entre F1 Scores

Como podemos ver, o TF-IDF + Naive Bayes foi o modelo que apresentou os melhores resultados, atingindo uma marca superior a 70 porcento tanto em accuracy como em F1 Score. Este desempenho supera significativamente as abordagens de Deep Learning e Embeddings, indicando que a importância estatística das palavras-chave é um preditor mais robusto para este conjunto de dados do que o contexto semântico

complexo do BERT (64 porcento) ou a representação vetorial do Word2Vec (53 porcento). A eficácia desta combinação clássica demonstra que, para este cenário específico, a identificação direta de termos frequentes e distintivos oferece uma generalização superior e uma classificação mais precisa do que modelos estruturalmente mais densos.

O tempo de processamento foi muito identico quer no TF-IDF + NB quer no Word2Vec. Contudo no Bert foi muito mais lento. Ou seja após avaliarmos os prós e contras chegamos que a melhor solução é o processo TF-IDF + NB.